Complete Theory — Data Analysis & Feature Transformation

Define-wise Notes • No Code, Formulas Where Needed • Syllabus 1.1 to 6.1

Customer Purchase Behaviour Analysis • Practical Exam (Set B) • 08 Aug 2026 • Ahmedabad (IST)

■ Complete Theory — Data Analysis & Feature Transformation

Define-wise Notes (No Code, Formulas Where Needed)

> ***Syllabus Coverage:*** *Data Analysis 1.1 → Feature Transformation 6.1*

---

1. Data Analysis

1.1 What is Data Analysis?

Definition: Systematic inspection, cleaning, transforming, and modeling of data to discover useful information, draw conclusions, and support decision-making.

Example: Analyzing 1,000 sales transactions to find which category (Home/Books) drives 38% revenue.

Types: Descriptive (what happened), Diagnostic (why), Predictive (what will happen), Prescriptive (what to do).

1.2 How to Plan a Data Science Project

Steps:

- **Business Understanding** — Define objective (e.g., predict churn)
- **Data Acquisition** — Identify sources (CSV, JSON, SQL, API)
- **Data Understanding & Cleaning** — Check quality, handle missing/outliers
- **EDA** — Univariate/Bivariate/Multivariate
- **Preprocessing & Feature Engineering** — Encoding, scaling, constructing features
- **Modeling** — Choose algorithm, train/test split
- **Evaluation** — Accuracy, RMSE, Precision/Recall
- **Deployment & Monitoring** — Put model in production, track drift

Key Principle: Plan FIRST (approach table) before coding — saves rework.

1.3 Framing a Machine Learning Problem

Definition: Translating business question into ML task.

Types:

- **Supervised:** Labeled data → **Classification** (predict category: will customer churn? Yes/No) vs **Regression** (predict number: spend ■?)
- **Unsupervised:** No label → **Clustering** (group customers), **Association**
- **Reinforcement:** Agent learns by reward

Example: “Predict amount per transaction” → Regression. “Classify payment_type” → Classification.

1.4 What are Tensors?

Definition: Multi-dimensional array — generalization of scalar/vector/matrix. Foundation of Deep Learning (PyTorch, TensorFlow).

Hierarchy:

- **Rank 0 / Scalar:** Single number `5`
- **Rank 1 / Vector:** 1D array `[1,2,3]` shape (3,)
- **Rank 2 / Matrix:** 2D array `[[1,2],[3,4]]` shape (2,2)
- **Rank 3+:** 3D+ e.g., RGB image shape (height, width, 3)

1.5 Tensor In-depth Explanation

Attributes:

- **Rank (ndim):** Number of axes
- **Shape:** Size per axis, e.g., (200, 6) for users table

- **Dtype:** Data type (int, float)

Operations: Addition, dot product, broadcasting (auto-expand smaller tensor), reshaping, transposition.

Why Important? Data in ML = tensors; e.g., users (200,6) is Rank 2 tensor, fed to model.

1.6 Working with CSV Files

Definition: Comma-Separated Values — plain text, rows = records, columns separated by `,`.

Example: `users.csv` → `user_id,name,age,gender,city`

Pros: Light, universal. **Cons:** No types, no nested data.

Handling: Check `header`, `delimiter`, `encoding`, parse dates.

1.7 Working with JSON / SQL

JSON (JavaScript Object Notation): Nested key-value, supports lists/objects. Example `sales.json` = `[{"transaction_id":"T001","amount":67.67},...]`. Good for APIs, hierarchical data.

SQL (Structured Query Language): Relational tables queried via `SELECT * FROM products WHERE price > 1000`. Example `inventory.sql` creates `products` table. Requires `sqlite3` / DB connection, executes script, then `SELECT`.

1.8 Fetching Data From an API

Definition: Application Programming Interface — URL endpoint returning data (usually JSON).

Example: `GET https://api.store.com/sales?city=Kolkata` → JSON. Needs `requests` library, handles `status_code`, `pagination`, `API key`, rate limits.

Steps: Request → Response (JSON) → Parse → DataFrame.

1.9 Understanding Your Data & Data Cleaning Process

Process: `head()` → `info()` (dtypes) → `describe()` (stats) → `isnull().sum()` → `duplicated()` → range checks (negative, future dates) → then Cleaning (imputation, outlier fix).

---

2. Exploratory Data Analysis (EDA)

Definition: Visual + statistical exploration to find patterns, anomalies, relationships before modeling.

2.1 EDA using Univariate Analysis

Definition: Analyze **one variable** at a time.

Methods: Histogram, boxplot, countplot, `mean/median/mode`, `skew`.

Example: Histogram of `amount` → right-skewed, mean 67.6 > median 56.4.

2.2 EDA using Bivariate Analysis

Definition: Relationship between **two variables**.

Methods: Scatter plot (`amount` vs `age`), barplot (`category` vs `amount`), correlation `r`, cross-tab.

Example: Scatter `frequency vs total_spent` → weak positive (r≈0.3) → frequent ≠ always high spend.

2.3 EDA using Multivariate Analysis

Definition: Three+ variables together.

Methods: Pairplot, heatmap (correlation matrix), groupby pivot (city × category × amount).

Example: Pivot `city × payment_type` revenue → Kolkata + Net Banking highest.

2.4 Pandas Profiling (ydata-profiling)

Definition: One-line auto EDA — generates HTML report with stats, correlations, missing, distributions, interactions.

Method: `ProfileReport(df, minimal=True).to_file("report.html")`

Output: Overview, variables, correlations, missing matrix — saves manual loops.

---

3. Missing Value Imputation

Definition: Filling gaps to avoid dropping rows (which loses information).

3.1 Handling Missing Numerical Data — SimpleImputer (Mean/Median)

Mean: `x■ = Σx / n` — use when numeric & roughly normal.

Median: Middle value — use when skewed (robust to outliers).

Example: `age` missing → fill with mean 31.

3.2 Handling Missing Categorical Data — SimpleImputer (Mode)

Most Frequent (Mode): Value with max count. Use for nominal categories.

Example: `gender` missing → fill with `Other` (most frequent).

3.3 Most Frequent Imputation (Missing Category Imputation)

Special case: When "Missing" itself is informative, create new category `"Missing"` or `"Unknown"` instead of imputing. Keeps signal that data was absent.

3.4 Missing Indicator + Random Sample Imputation

Missing Indicator: Add binary column `is_missing = 1 if null else 0` → model learns missingness pattern.

Random Sample: Fill with random observed value from same column — preserves distribution better than single mean (avoids variance shrinkage).

3.5 KNN Imputer (Multivariate Imputation)

Definition: Fill using **k nearest neighbours** (Euclidean distance on other features).

Formula: For missing `x_i`, `x■_i = mean(k nearest neighbours' values)` where distance `d = √(Σ(x_a - x_b)²)`.

Example: Age missing for user with `city=Jaipur, tenure=1150` → average age of 5 most similar Jaipur users.

When: Relationships exist between features. **Pros:** smarter than mean. **Cons:** heavy on large data.

3.6 Multivariate Imputation by Chained Equations (MICE Algorithm)

Definition: Iterative method — each column with missing is modeled as function of other columns, cycle repeats.

Steps: 1) Init mean → 2) For each column, regress on others → predict missing → 3) Repeat 10+ cycles.

Assumption: MAR. **Pros:** Most accurate, preserves correlations. **Cons:** Slow, assumes model is correct.

Example: Missing `price` predicted from `category + stock` via regression, iteratively refined.

---

4. Outliers in Machine Learning

Definition: Observation distant from others; can distort mean/scaler/model.

4.1 Z-Score Method

Formula: `z = (x - μ) / σ`

Rule: Outlier if `|z| > 3` (≈99.7% within 3σ for Normal).

Steps: Compute μ, σ → calculate z → flag >3 → remove/cap.

Example: Amount ■`550`, μ=67.6, σ=45.4 → `z=(550-67.6)/45.4=10.6` → outlier.

Pros/Cons: Simple, but assumes Normal — fails on skewed sales (found only 11 vs IQR 44).

4.2 IQR Method

Formula: `IQR = Q3 - Q1`

`Lower = Q1 - 1.5×IQR`, `Upper = Q3 + 1.5×IQR`

Outlier if `x < Lower` or `x > Upper`.

Example: Q1=37.7, Q3=82.9, IQR=45.2 → Bounds `[-30, 150]` → amount >150 flagged (44 rows).

Pros: No normality need, robust.

4.3 Percentile Method

Formula: Outlier if `x < Pk` or `x > P(100-k)`, e.g., `k=5` → less than 5th percentile or greater than 95th.

Example: 5th=22.65, 95th=139.02 → values outside capped. Simple, intuitive, no stats needed.

4.4 Winsorization Technique

Definition: Capping extremes at percentiles instead of deleting.

Formula: `x_winsor = min(max(x, P5), P95)`

Example: ■`550` → ■`139.02 (P95)`, ■`7` → ■`22.65 (P5)`.

Why not delete? Keeps all 1000 rows, preserves ranking, reduces skew `1.35 → 0.14` after log.

---

5. Feature Transformation & Construction (6.1)

5.1 Handling Mixed Variables

Definition: Dataset has numeric + categorical + dates. Must process each type separately via **ColumnTransformer** (applies different pipelines per column).

5.2 Handling Date and Time Variables

Method: Extract `day, month, year, dayofweek, is_weekend, quarter` from `datetime`.

Example: `2025-10-04` → `month=10, is_weekend=1` → learns seasonality/weekend effect.

5.3 Complete Case Analysis (Handling Missing Data)

Definition: Delete any row with missing (listwise deletion).

Formula: Keep only `df.dropna()`.

Pros: Simple. **Cons:** Loses data, biased if not MCAR. Used only if <5% missing. Not chosen in our project (0% missing but imputation kept for production).

5.4 Encoding Categorical Data

a) Label Encoding: Assign integers `0,1,2...` per category.

Example: `gender: Female 0, Male 1, Other 2` → 1 column. Use for binary/tree models; risks false order.

b) Ordinal Encoding: Like Label but order matters.

Define categories `['Tier3','Tier2','Tier1']` → `0<1<2`. Example: `city_tier` Tier1 is premium → ordered. Use `OrdinalEncoder(categories=[...])`.

c) One-Hot Encoding: Create binary column per category.

Example: `payment_type: Cash,UPI,Wallet...` → 6 columns `pay_Cash=0/1`. No false order. Use for nominal. Cost: dimensionality ↑.

5.5 Encoding Numerical Features

a) Binning (Discretization): Group continuous into intervals.

Uniform: equal width `[0-40,40-80,80-600]` → labels `Low/Med/High`. Business-friendly.

b) Binarization: Threshold → 0/1. Formula: `x' = 1 if x > threshold else 0`. Example: `is_high_spend = 1 if amount>100`.

c) Quantile Binning: Equal **frequency** per bin (percentiles). Example: 3 bins each 33% rows → handles skew better than uniform.

d) K-Means Binning: Cluster values via K-Means → bins are clusters. Captures natural groups better than fixed width.

5.6 Feature Scaling

Why? Features on different scales dominate distance.

a) Standardization (Z-score): `z = (x - μ)/σ` → mean 0, std 1, unbounded. Use for KNN, SVM, PCA.

Example: age 35, μ=31, σ=7.2 → `z=0.55`.

b) Normalization: Often means scaling to 0-1 (see MinMax) or unit vector `x/||x||`.

c) MinMaxScaling: `x' = (x - min)/(max - min)` → range [0,1]. Use for Neural Nets, image.

Example: amount 67, min 22, max 139 → `(67-22)/117=0.38`.

d) MaxAbsScaling: `x' = x / max(|x|)` → range [-1,1], preserves sparsity (doesn't shift 0). Good for sparse data.

e) RobustScaling: `x' = (x - median)/IQR` → robust to outliers. Use when outliers remain.

Comparison: Standard sensitive to outliers; MinMax sensitive; Robust handles.

5.7 Feature Construction & Splitting

Construction: Create new features from existing. Example: `avg_monthly_spend = total_spent / active_months`, `tenure = max_date - registration_date`, `weekend_ratio`.

Splitting: Reverse — split composite into parts. Example: `date → day, month, year`.

5.8 FunctionTransformer (Log, Reciprocal, Square Root)

Definition: Apply mathematical function via `FunctionTransformer`.

- **Log:** `x' = log(1+x)` or `ln(x)` — reduces right skew, handles 0. Example: skew `1.35 → 0.14`.
- **Reciprocal:** `x' = 1/x` — inverts, useful for rates.
- **Square Root:** `x' = √x` — milder than log, for moderate skew.

5.9 PowerTransformer (Box-Cox & Yeo-Johnson)

Box-Cox: `x(λ) = (x^λ -1)/λ if λ≠0 else log(x)` → requires `x>0`. Finds optimal λ to make normal.

Yeo-Johnson: Similar but works for **any** x (including 0/negative): piecewise with λ.

Use: Auto-normalize skewed features better than manual log.

5.10 Column Transformer

Definition: Applies different transformers to different columns in one pipeline.

Example: `ColumnTransformer([('num', StandardScaler(), ['age','amount']), ('cat', OneHotEncoder(), ['payment_type'])])` → numeric scaled, categorical one-hot, simultaneously. Essential for mixed data.

Summary Table — When to Use What

Problem	Method	Formula / Rule
Missing numeric	Mean/Median	`mean` if normal, `median` if skewed
Missing categorical	Mode / Missing category	Most frequent or “Missing”
Multivariate missing	KNN / MICE	Neighbours or chained regressions

| Outlier normal | Z-score | `\|z\|>3` |

Outlier skewed	IQR / Percentile	`1.5×IQR` or `P5/P95`
Keep data but tame	Winsorization	`clip(P5,P95)`
Date	Extract parts	`day, month, year`
Category nominal	One-Hot	6 cols
Category ordinal	Ordinal	`Tier3 0 < Tier2 1 < Tier1 2`
Continuous → groups	Binning	`Low/Med/High`
Skew	Log / Box-Cox	`log1p`, `(x^λ-1)/λ`
Scale distance model	Standardize	`(x-μ)/σ`
Scale NN	MinMax	`(x-min)/(max-min)`

| Sparse/outlier | MaxAbs / Robust | `x/max\|x\|`, `(x-median)/IQR` |

End of Theory — covers syllabus define-wise with formulas. Ready for exam viva & write-up.